Desarrollo inicial del sistema de votaciones

Festival de la Leyenda Vallenata 2027

Entrega 2

Esteban Villada Henao

Fundación Universitaria del Área Andina

Programa académico: pendiente de confirmar

Desarrollo Web

Docente: DEIVYS MORALES URIBE

26 de octubre de 2026

Introducción y objetivo

La segunda entrega convierte el diseño inicial en una aplicación Django MVT conectada a MySQL. Implementa autenticación, administración del catálogo y consulta pública dinámica. Las ediciones, categorías, votaciones y opciones son datos ficticios; la emisión de votos corresponde a etapas posteriores (*Entrega 2 - Festival Vallenato 2027*, 2026).

Objetivo: construir y verificar una base funcional, persistente y ejecutable con un comando, manteniendo separadas la presentación, las reglas y el almacenamiento.

Tabla 1

Estructura y relaciones implementadas

Componente	Responsabilidad
config	Configuración MySQL, rutas raíz y WSGI.
accounts	Registro, inicio y cierre de sesión mediante Django User.
catalog	Modelos, vistas ORM, administración y migración 0001_initial.
templates / static	Interfaz responsive en español y estilos locales.
Edición → Categoría → Votación → Opción	Claves foráneas de uno a muchos; nombres únicos por contexto.

Las vistas consultan el ORM y entregan objetos a plantillas. Django Admin permite modificar los datos sin construir otro panel. Los borradores no se muestran públicamente ni se recuperan por acceso directo a su URL.

Desarrollo técnico

MySQL utiliza utf8mb4 y modo estricto. Django conserva validadores y hash de contraseñas, sesiones y protección CSRF. El registro público no concede privilegios; cerrar sesión exige POST. Las redirecciones de acceso se limitan a destinos seguros (Django Software Foundation, s. f.-a, s. f.-c).

Fragmentos del código implementado

```
# catalog/models.py
class Category(models.Model):
    edition = models.ForeignKey(Edition, on_delete=models.CASCADE,
    related_name="categories")
    name = models.CharField(max_length=120)
    description = models.TextField(blank=True)

    class Meta:
        ordering = ["name"]
        constraints = [
            models.UniqueConstraint(fields=["edition", "name"],
    name="unique_category_per_edition")
        ]
        verbose_name = "categoría"
        verbose_name_plural = "categorías"

    def __str__(self):
        return f"{self.name} ({self.edition.year})"

# accounts/views.py
class FestivalLogoutView(LogoutView):
    http_method_names = ["post", "options"]
    next_page = reverse_lazy("catalog:home")
```

Las migraciones versionadas crean tablas y restricciones. El arranque aplica migrate, crea datos de prueba de forma condicional y recopila estáticos. No genera migraciones nuevas ni borra datos al reiniciar (Django Software Foundation, s. f.-b).

Ejecución y pruebas

Con Docker y Compose instalados, ejecutar **./start.sh** desde la carpeta del proyecto. El comando genera secretos locales si faltan, construye imágenes y espera MySQL antes de iniciar Django. Aplicación: http://localhost:8000/. La primera descarga requiere internet (Docker Inc., s. f.).

Tabla 2

Comprobaciones observadas

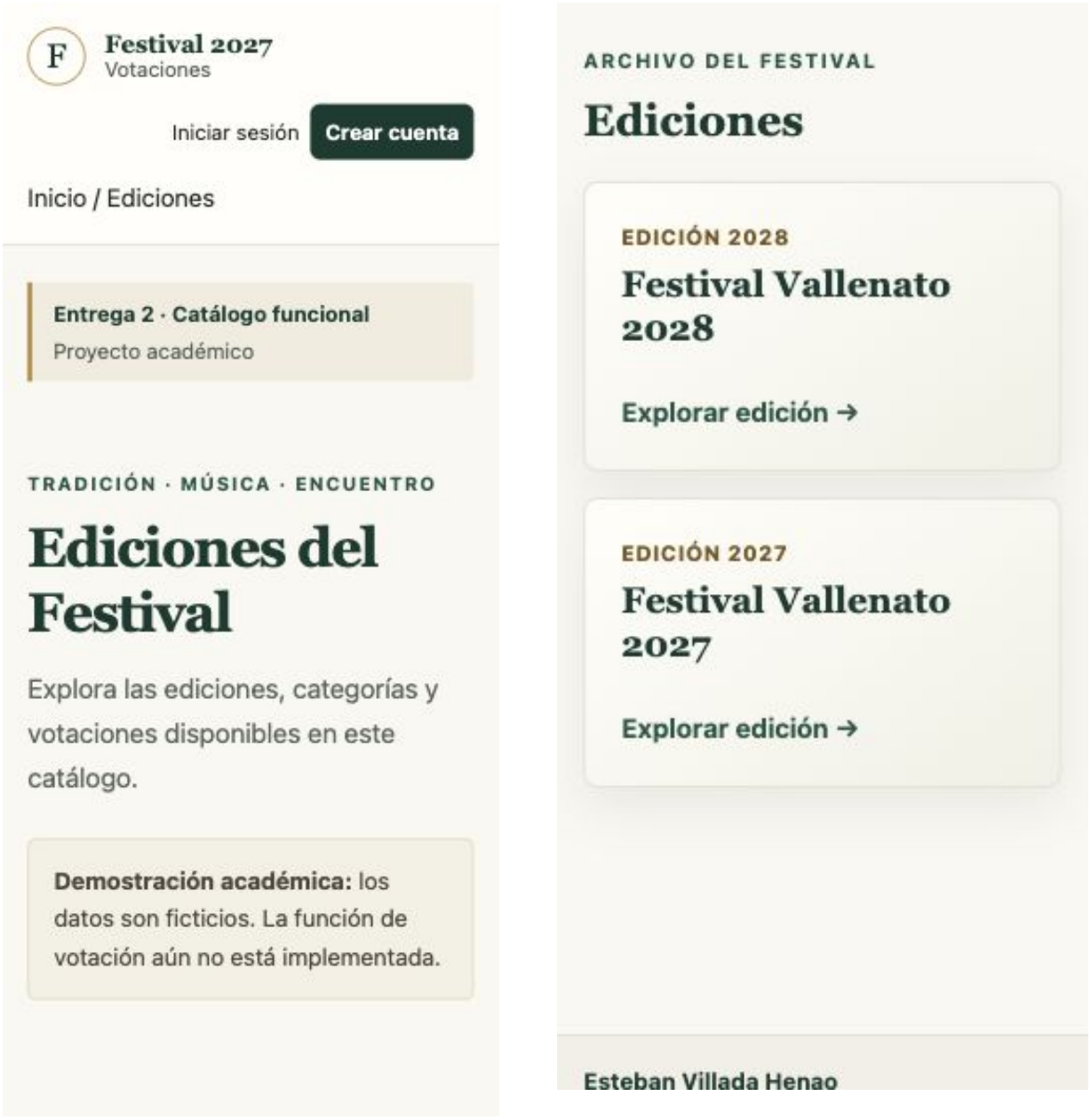
Prueba / procedimiento	**Resultado**
python manage.py test --noinput	23 pruebas aprobadas con MySQL real; base aislada test_festival.
check; makemigrations --check --dry-run	Sin errores de configuración; modelos y migraciones coinciden.
showmigrations	Migraciones de catálogo, autenticación, Admin y sesiones aplicadas.
SELECT VERSION(); consultas ORM	MySQL 8.4.13; dos ediciones y catálogo persistido.
Registro, login, logout y Admin en navegador	Cuenta ficticia creada; sesión iniciada/cerrada; categoría guardada.
Detener web y ejecutar ./start.sh	Mismos registros y categoría de evidencia con ID 3: persistencia aprobada.
Inspección DOM móvil	Ancho efectivo 360 px; sin desbordamiento horizontal.

El contenedor web no recibe la contraseña root de MySQL y se ejecuta sin root. La base no publica puertos; la aplicación se enlaza a localhost. Los secretos permanecen en .env, excluido del repositorio, del ZIP y de las capturas. No es un despliegue productivo.

Evidencia: consulta de ediciones

Figura 1

Catálogo público con ediciones almacenadas en MySQL

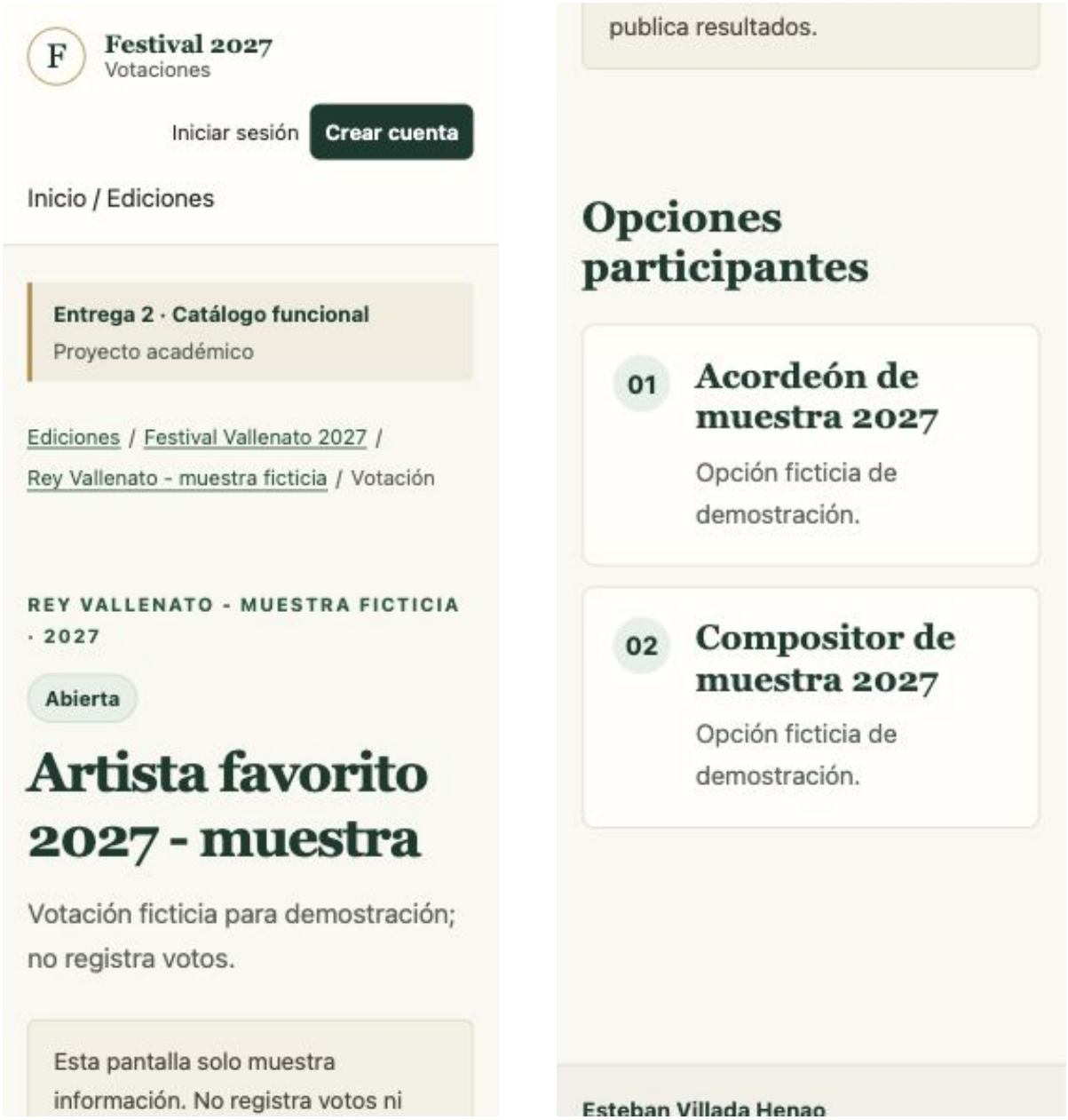


Nota. Fragmentos superior e inferior de una misma captura real. Se observan las ediciones 2027 y 2028 y el aviso de datos ficticios. El contenido proviene del ORM, no de arreglos estáticos.

Evidencia: votación y opciones

Figura 2

Recorrido público hasta las opciones de una votación

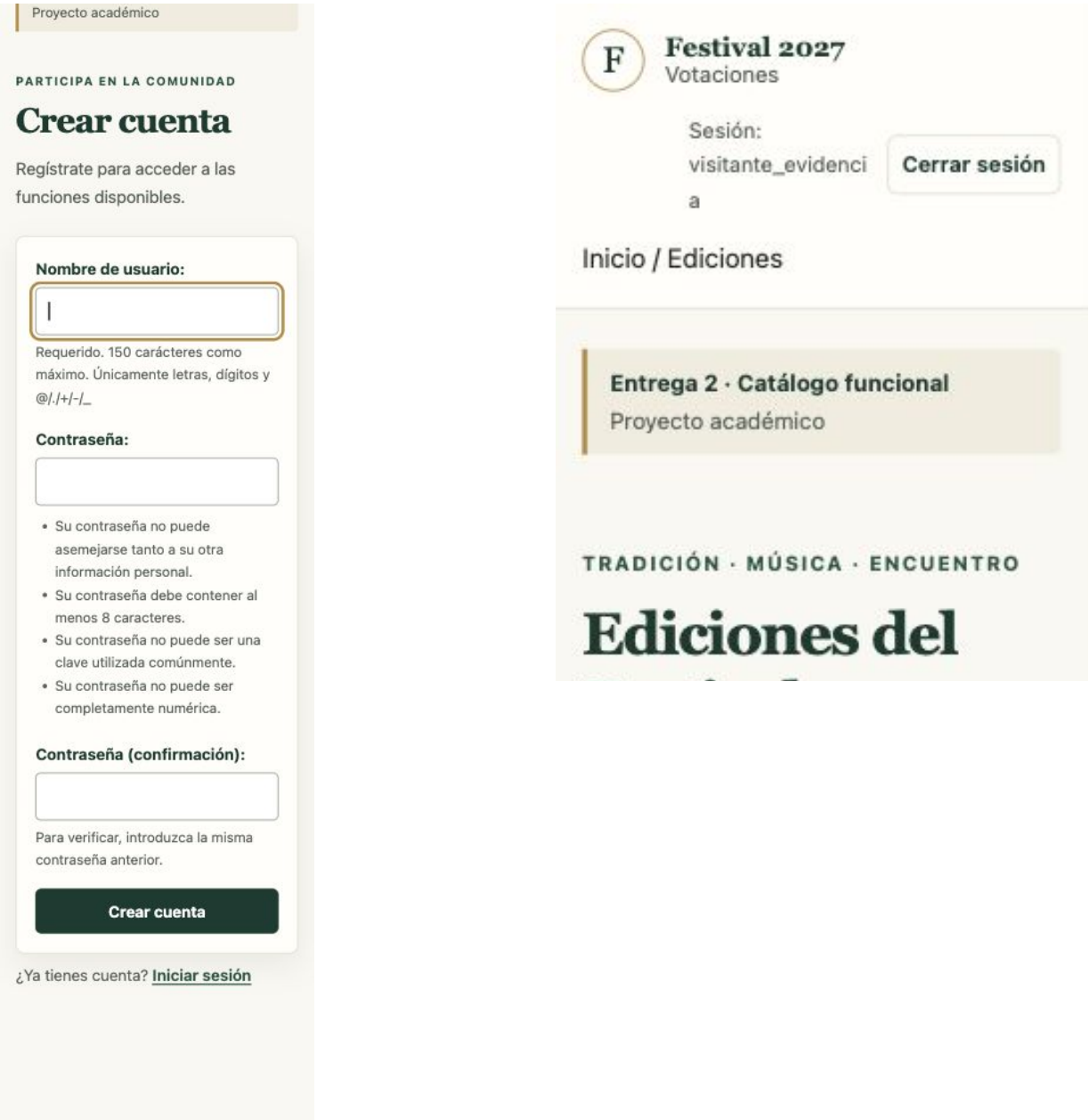


Nota. Dos fragmentos de la misma pantalla. La ruta conserva edición y categoría; se muestran estado, descripción y opciones relacionadas. La interfaz declara que todavía no registra votos.

Evidencia: autenticación

Figura 3

Registro y sesión de un usuario sin permisos administrativos

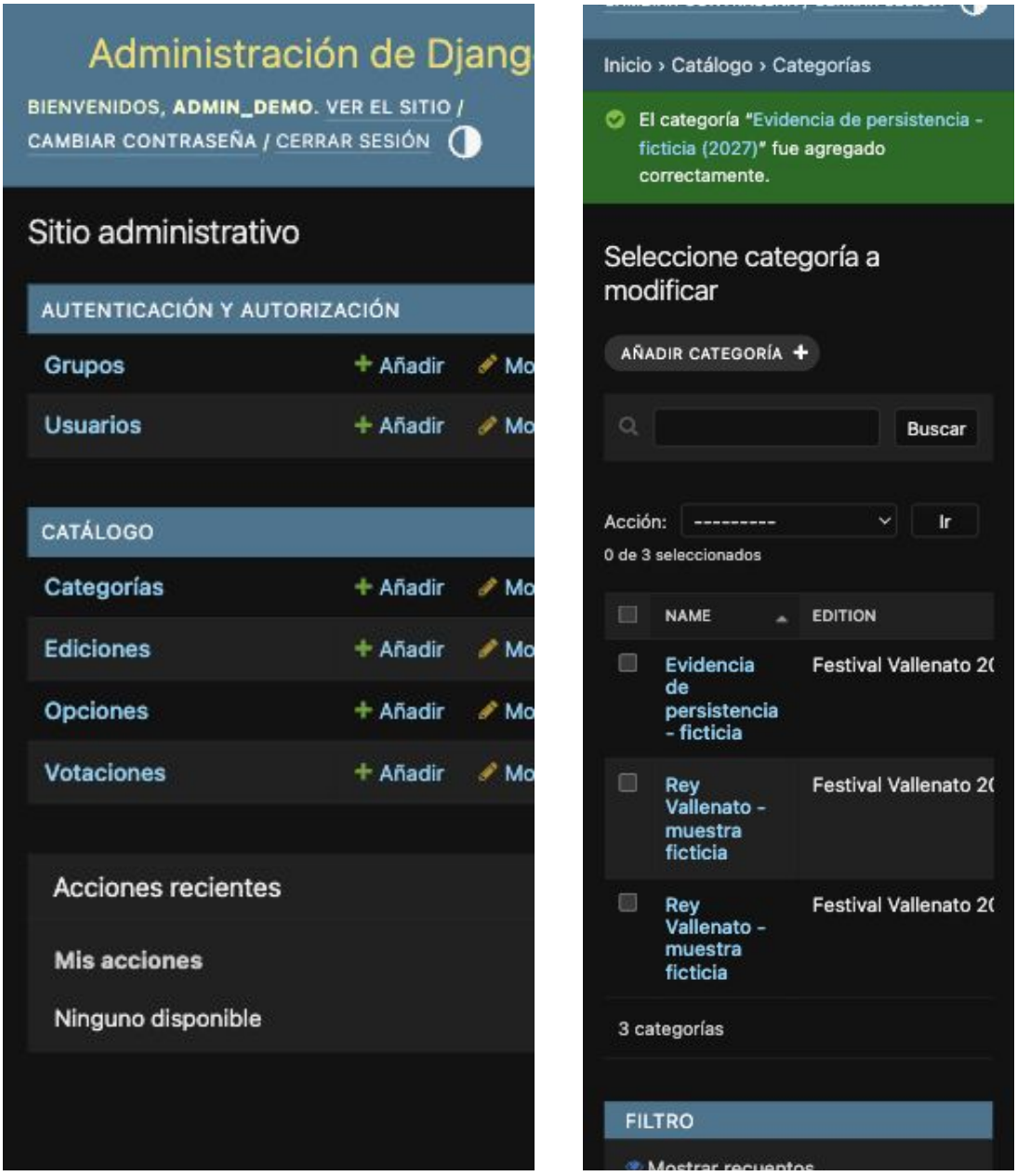


Nota. Izquierda: formulario real con validadores de contraseña. Derecha: sesión de visitante_evidencia iniciada tras el registro y login; no aparece el enlace administrativo. La prueba posterior de logout devolvió los enlaces de acceso. No se muestran contraseñas.

Evidencia: administración

Figura 4

Modelos administrables y confirmación de guardado



Nota. Izquierda: Django Admin reconoce ediciones, categorías, votaciones y opciones. Derecha: mensaje de guardado y nueva categoría ficticia vinculada a 2027. El cambio se realizó mediante el formulario, no modificando la base manualmente.

Evidencia: visualización dinámica y persistencia

Figura 5

La categoría registrada en Admin aparece en la interfaz pública



Nota. Fragmentos de la consulta pública de 2027. Aparece Evidencia de persistencia - ficticia, creada desde Admin. Después de detener web y arrancar con ./start.sh se conservaron tres categorías, cuatro votaciones y cuatro opciones; la categoría mantuvo ID 3.

Repositorio, video y conclusiones

Repositorio público: https://github.com/wesvh/web-dev-areandina-2027. Código, migraciones, pruebas y evidencias publicados en la rama master mediante SSH de la cuenta personal wesvh. La publicación pública fue autorizada expresamente el 26 de octubre de 2026. El README explica la ejecución local con Docker; publicar el código no implica alojar la aplicación en internet.

Video: enlace pendiente de proporcionar. Se incluye un guion de grabación en el proyecto; no se inventó una demostración grabada ni la participación de otras personas. Antes del envío, publicar el video con permisos para el docente y reemplazar este pendiente.

Se logró una base ejecutable con relaciones persistentes, autenticación nativa, administración y consulta dinámica. Las pruebas y el reinicio respaldan el alcance funcional; no prueban seguridad productiva, carga ni reglas definitivas de votación.

Responsabilidades: Esteban Villada Henao figura como único autor, responsable de revisar y justificar análisis, código, pruebas y evidencias. Se utilizó IA como apoyo técnico. La guía exige cuatro integrantes y establece 0/5 en el criterio de trabajo grupal para entregas individuales; esta presentación no declara una excepción aprobada.

Antes de entregar: completar programa académico y enlace del video; verificar permisos del video desde otra sesión. El repositorio es público y no requiere invitación para consultar el código. Registro definitivo de votos, resultados, elegibilidad avanzada y producción quedan para próximas fases.

Referencias

Django Software Foundation. (s. f.-a). *Databases*. https://docs.djangoproject.com/en/5.2/ref/databases/

Django Software Foundation. (s. f.-b). *Migrations*. https://docs.djangoproject.com/en/5.2/topics/migrations/

Django Software Foundation. (s. f.-c). *Using the Django authentication system*. https://docs.djangoproject.com/en/5.2/topics/auth/default/

Docker Inc. (s. f.). *Control startup and shutdown order in Compose*. https://docs.docker.com/compose/how-tos/startup-order/

Entrega 2 - Festival Vallenato 2027. (2026). [Guía académica suministrada por el docente]. Fundación Universitaria del Área Andina.